DECENTRATHON 6.0 · ТРЕК AI 1 · SOUP KERNEL CHALLENGE

Свои ядра Triton для 4-битного дообучения на бесплатной T4

Тот же прогон soup train: в 1,3 раза быстрее и в 2,4 раза меньше памяти при том же качестве

1,305×
быстрее базы
86,27 → 66,10 с

2,440×
меньше пик памяти
14 345 → 5 879 МиБ

+0,000%
к eval loss базы
2,04962 = 2,04962

10 / 10
проверок harness
70 своих + 79 тестов Soup

Команда ID Automation · Даниэль Искрин

31 октября 2026

ПОСТАНОВКА

Ускорить чужое обучение, не меняя в нём ничего

ЗАДАЧА

Qwen2.5-1.5B-Instruct, веса NF4 + LoRA r=16

64 шага, батч 4 × до 512 токенов

256 казахских инструкций

Одна Kaggle T4: 15 ГБ, fp16, без bf16

НЕЛЬЗЯ

Менять модель, данные, батчи, шаги, LoRA, lr

Править harness, tests, task, настройки замера

Готовые библиотеки: unsloth, liger, flash-attn

Встроенные режимы: packing, checkpointing

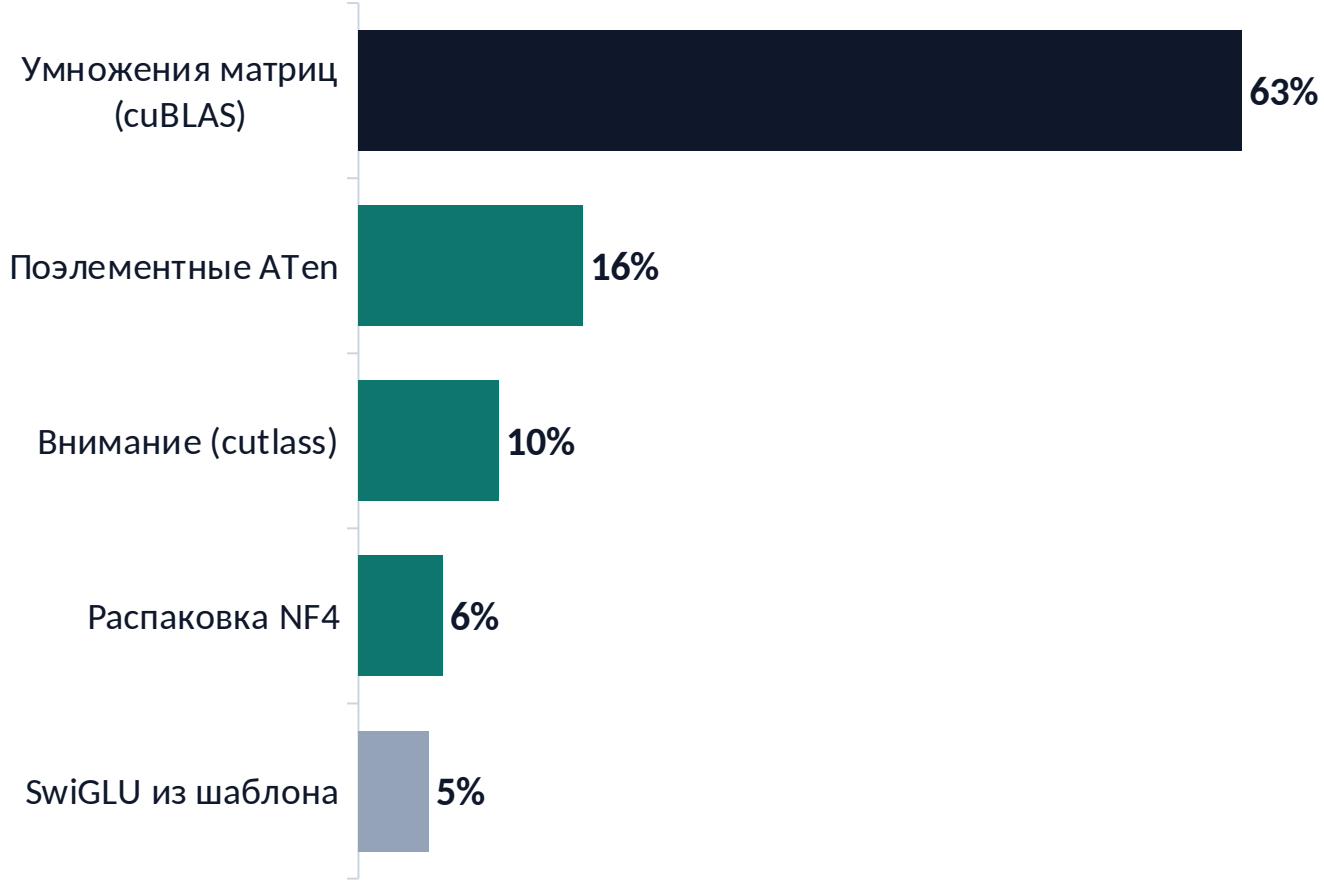
УСЛОВИЯ ЗАЧЁТА

Свои @triton.jit из solution/ — не меньше 5% времени обучения. Время, память и loss — не хуже базы.

База: 86,27 с · 14 345 МиБ · eval loss 2,04962

ДИАГНОСТИКА

63% GPU-времени — умножения матриц, и их не обогнать



СТРАТЕГИЯ

cuBLAS на T4 уже на пределе — матмулы не трогаем.

Бьём по всему вокруг: лишние fp32-копии и касты, плитки внимания над диагональю, распаковку NF4 своим ядром.

Профиль: 16 шагов torch.profiler.

РЕШЕНИЕ

Восемь своих ядер в каждом шаге обучения

Ядро	Что заменяет	Доля
ida_nf4_dequant_kernel	распаковку NF4 в forward и backward	5,09%
fused_rmsnorm_fwd / bwd	Qwen2RMSNorm	0,72% / 0,96%
ida_attn_prologue_fwd / bwd	RoPE + fp16 + разворот GQA	0,58% / 0,54%
ida_entropy_argmax_kernel	энтропию и argmax без копии логитов	0,50%
fused_ce_fwd / bwd	кросс-энтропию кусками словаря	0,28% / 0,44%
Итого (проверка №4)	меньшая из двух долей; порог 5%	9,1%

БЕЗ НОВЫХ ЯДЕР

Эмбеддинг и lm_head хранятся в fp16

MLP целиком в fp16

Логиты не поднимаются в fp32

Компиляция ядер параллельно с загрузкой модели

САМОЕ ТЯЖЁЛОЕ ЯДРО

Распаковка NF4: 392 вызова за шаг, 5,09% времени

1 · БАЙТ



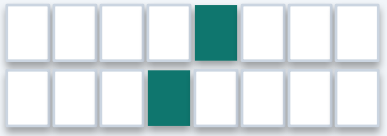
два 4-битных значения: старший нибл — первый

2 · ДВА НИБЛА



индексы 0…15, раскладка tl.join

3 · ТАБЛИЦА NF4



16 фиксированных уровней (QLoRA)

4 · × МАСШТАБ БЛОКА

w = code[nibble] · absmax

один масштаб на 64 значения, результат сразу в fp16

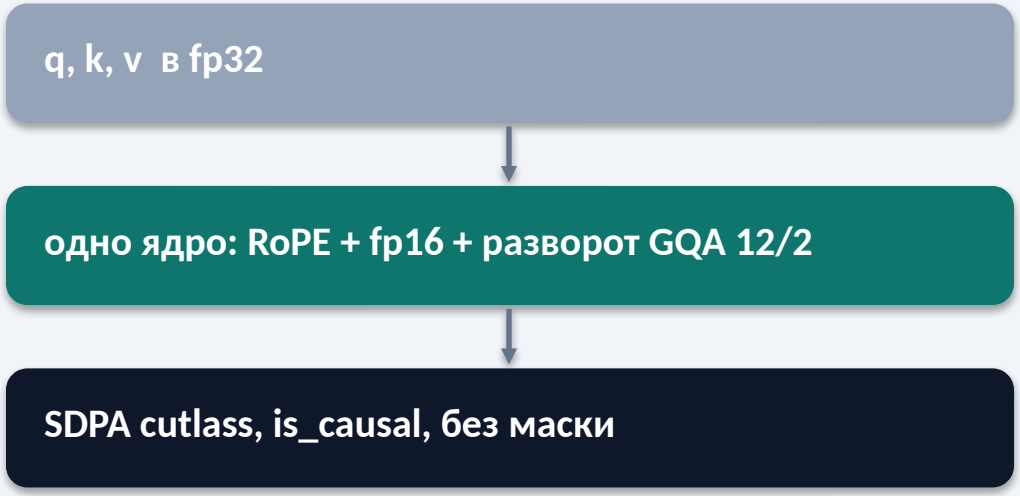
ДВОЙНОЕ КВАНТОВАНИЕ — ВНУТРИ ЯДРА

absmax[j] = code2[absmax8[j]] · absmax2[j / 256] + offset

Побитно совпадает с bitsandbytes (тест на реальном quant_state). На T4 при M > 512 bitsandbytes распаковывает веса и в forward, поэтому ядро работает дважды за шаг: 24 897 запусков, 2 061,5 мс.

Внимание без лишних плиток, логиты — в одном буфере

ПРОЛОГ ВНИМАНИЯ + is_causal



TRL дополняет батч справа: реальный токен и так не видит паддинг. Guard проверяет это каждый шаг.

ЛОГИТЫ НА ПИКЕ: ОДИН БУФЕР ВМЕСТО ДВУХ

Было



Стало



Энтропия и argmax — одним проходом нашего ядра, без копии логитов. Градиент CE пишется в буфер самих логитов.

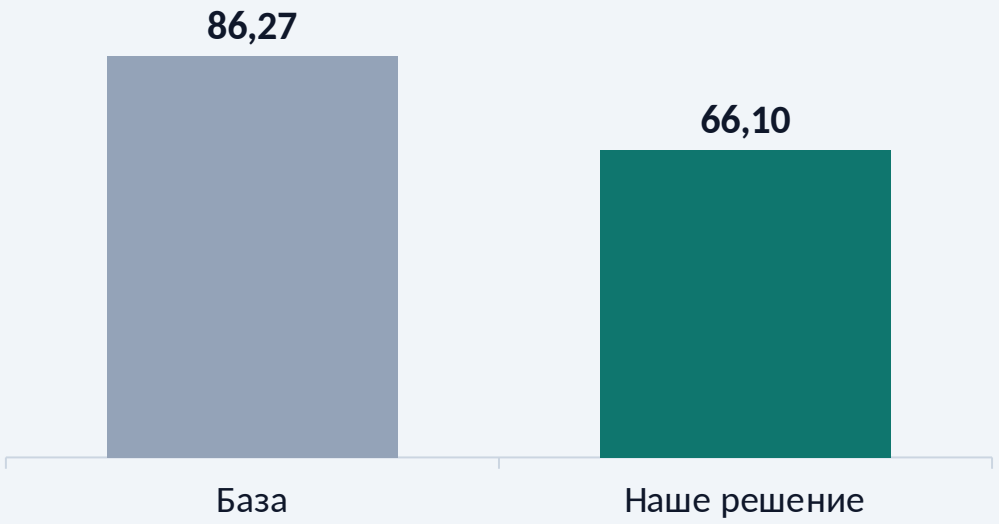
РЕЗУЛЬТАТЫ

На 20 секунд быстрее и в 2,4 раза легче по памяти

ВРЕМЯ soup train, с (медиана)

1,305×

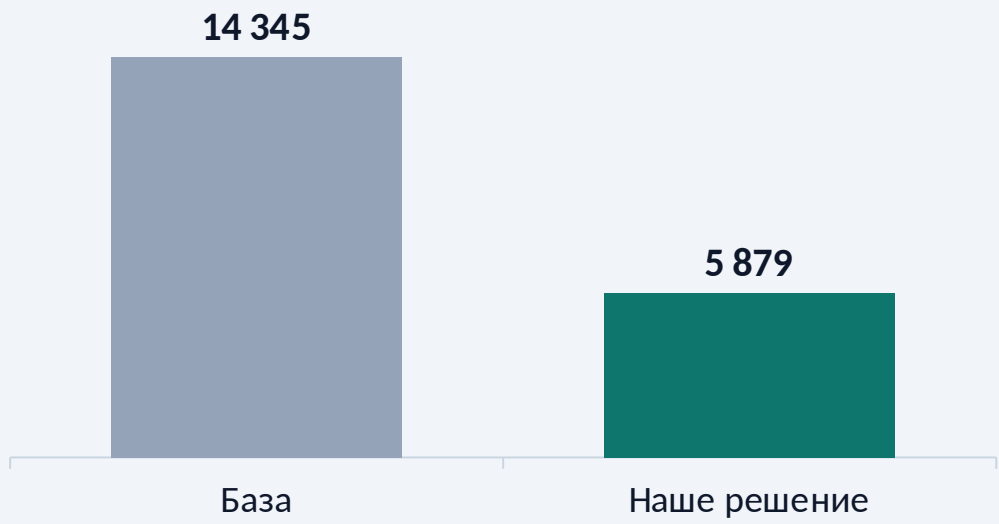
цикл обучения 58,57 → 42,30 с (1,385×)



ПИК ПАМЯТИ GPU, МиБ

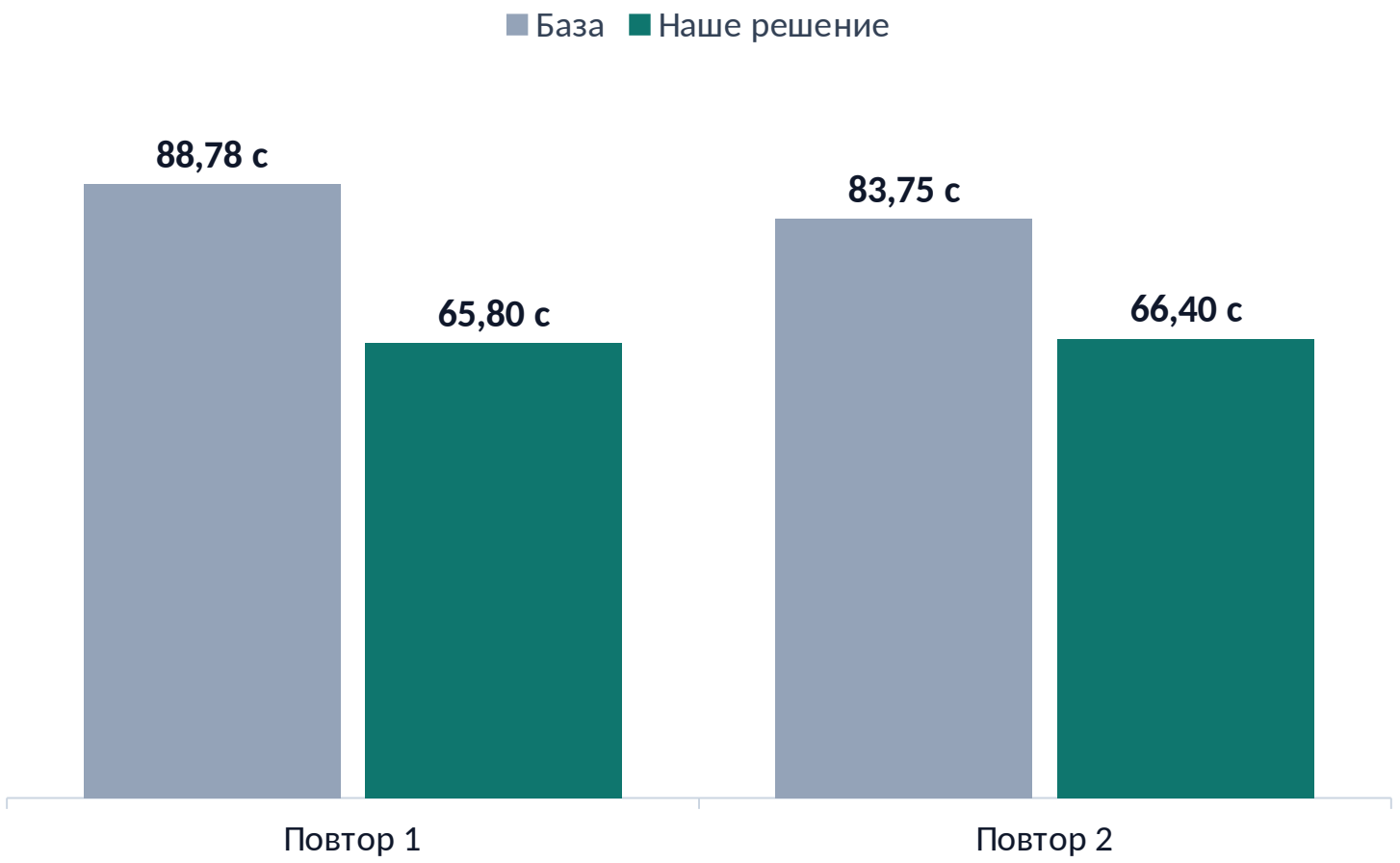
2,440×

eval loss 2,04962 = 2,04962 (+0,000%)



СТАБИЛЬНОСТЬ

Решение стабильнее базы: разброс 0,9% против 5,8%



БАЗА

5,8%

88,78 / 83,75 с
разница 5,03 с

НАШЕ РЕШЕНИЕ

0,9%

65,80 / 66,40 с
разница 0,60 с

ПРОВЕРКИ

Всё, что требует harness, пройдено без исключений

10 / 10

проверок check_rules.py

ALL CHECKS PASSED

9,1%

доля своих ядер

порог 5%

+0,000%

к eval loss базы

допуск +0,5%

70 + 79

своих тестов + тестов Soup

все проходят

ОГРАНИЧЕНИЯ

Что не сработало и что мы сознательно не делали

Что пробовали	Результат	Почему выключено
Своё ядро внимания целиком: online-softmax, GQA, маски	на реальных данных loss 2,32 вместо 2,05	ошибки на строках паддинга, ~30× медленнее cutlass
Своё умножение на словарь 940 × 1536 × 151 936	тесты против F.linear проходят	lm_head не квантован, оставлен cuBLAS
Не считать строки паддинга	оценка: −8…−14 с	риск попасть под запрет packing / padding_free
Свой 4-битный GEMM, cuBLASLt, слияние q/k/v	0…1% по оценке	ниже разброса замера

Дальше: внимание без атомарных сложений (FlashAttention-2), хранение k/v на 2 головах и пересчёт входа LoRA в backward.

ИТОГ

Тот же результат обучения на карте, которая есть у каждого

1,305× быстрее

86,27 → 66,10 с по протоколу жюри

2,440× меньше памяти

5 879 МиБ на пике вместо 14 345

Воспроизводимо

публичный ноутбук, README, 70 тестов

Переносимо

NF4, fp16 вместо fp32-копий, пролог внимания

kaggle.com/code/danieliskrin/notebookd124ebce8b

github.com/BAITC-Hacks/hack-8539eaba-id-automation · зеркало: github.com/danieliskrin30-lang/soup-kernel-v7